AEGIS-AT v2: Does Sender-Constraint Recover Attribution?

Measuring DPoP, Chain Depth, and Log Integrity in Multi-Agent Delegation — and Confirming the Attribution Gap Is Structural

Rahul Singh*

June 2026

Abstract

The v1 AEGIS-AT benchmark [1] showed that adding OAuth 2.0 Token Exchange (RFC 8693) delegation to a correctly-functioning multi-agent SOC pipeline makes audit attribution *worse*: the Attribution Integrity Score (AIS) rises to 1.0 under per-agent identity, then regresses to 0.0 once signed delegation is added, and tamper-evident logging does not recover it. v1 conceded four limitations and named one fix it did not measure. This paper, **AEGIS-AT v2**, retires those four limitations and measures the named fix. We make five changes over a single, shared codebase. **(1)** A real process-boundary ground-truth recorder (`multiprocessing` + `os.getpid()`) replaces v1's thread-name proxy; PID-based attribution is unspoofable by an in-process rename. **(2) Baseline 5** adds sender-constrained tokens via DPOP (RFC 9449): the executor must hold the key its token is bound to. AIS recovers to **1.0** — the conjectured fix works. **(3)** A real hash-chained, signed tamper-evident log and a new **Log Integrity Score** (LIS) make Baseline 4 concrete; LIS reaches 1.0 (every tamper detected) while AIS stays 0.0 — a tamper-evident record of the *wrong* actor. **(4)** A second topology T2 (a 3-agent re-delegation chain) shows the gap *does not heal with chain depth*: the curve is identical to the 2-agent T1, and at depth 2 the claimed actor is the deepest requester, never the executor. **(5)** A stochastic Bernoulli(p) escalation policy with Wilson 95% intervals and adaptive sample-size escalation retires v1's degenerate confidence intervals and confirms the curve shape is invariant to attack frequency. Every predicted value is pre-registered in a SHA-256-locked threat model before the measuring code was written, and all figures are generated from the live harness. The result strengthens v1's claim: the attribution gap is structural, sender-constraint is the layer that closes it, and tamper-evidence and chain depth are orthogonal to it.

Keywords: multi-agent systems, AI agent security, attribution, non-repudiation, OAuth 2.0 Token Exchange, RFC 8693, DPoP, RFC 9449, sender-constrained tokens, proof-of-possession, confused deputy, audit logging, benchmark.

Contents

1 Introduction **2**

*Independent researcher, Jersey City, NJ. Correspondence: rahul.rs1397@gmail.com · github.com/rahulsingh1397. This paper extends the v1 benchmark [1]; the v1 artifact is frozen at tag `v1.0.0`.

2 Background and Relationship to v1 **4**
2.1 Sender-constrained tokens: DPoP 4
2.2 What is inherited, and the one thing that moved 5
2.3 OAuth 2.1: the gap is version-independent 5
2.4 Proposed bindings, not measured baselines 5

3 Process-Boundary Ground Truth **6**

4 Baseline 5: Sender-Constrained Tokens (DPoP) **6**
4.1 Mechanism 6
4.2 Why it recovers the curve 7
4.3 What the binding does and does not guarantee 7

5 Hash-Chained Log and the Log Integrity Score **7**

6 Topology T2: A Three-Agent Chain **8**

7 Stochastic Policy and Confidence Intervals **9**

8 Implementation **9**

9 Experimental Methodology **10**

10 Results **10**
10.1 The two curves 10
10.2 Log integrity 10
10.3 Stochastic confidence intervals 11

11 Discussion **11**

12 Limitations and Validity Threats **12**

13 Future Work **13**

14 Conclusion **13**

Reproducibility and Artifact Availability **13**

References **14**

1 Introduction

The v1 result, in one line. AEGIS-AT v1 [1] built the smallest multi-agent system in which sibling impersonation can occur — two agents (`Agent-Enrich`, read-only; `Agent-Contain`, write-capable) sharing one scope-gated tool — and measured attribution correctness under a sibling-impersonation attack across four defense baselines applied as configuration flags over a single codebase. The measured curve was non-monotonic: AIS = 1.0 under per-agent identity (B2), collapsing to AIS = 0.0 once RFC 8693 delegation is added (B3), and staying at 0.0 under tamper-evident logging (B4). The cause is structural: RFC 8693's current actor (`act.sub`) names the agent

that *requested* delegated authority, while in a multi-agent hand-off the agent that *executes* the action can differ, and the standard provides no field for the executor.

What v1 left open. v1 was deliberately bounded and conceded four limitations plus one unmeasured fix: (a) a thread-name proxy for process identity; (b) a Baseline 4 that was attribution-only (the hash-chained log was not built); (c) a single topology (one tested configuration); (d) categorical AIS ($\in \{0, 1\}$) with degenerate confidence intervals; and the named-but-unmeasured fix, (e) sender-constrained tokens (a hypothetical "Baseline 5"). This paper closes all five.

The five changes and the headline. Each change is a flag or module over the *same* codebase, so the new numbers remain comparable to v1's (Table 1). The headline: **sender-constraint (dpop) recovers attribution to ais = 1.0 at Baseline 5**, confirming v1's conjecture; and the three robustness changes — a real process boundary, a real tamper-evident log, and a second topology — all leave the v1 finding standing rather than weakening it. The non-monotonic curve, now five points and measured on two topologies, is shown in Figure 1.

Table 1: The v2 Attribution Integrity Score across five baselines, measured identically on both topologies (T1: 2-agent; T2: 3-agent). Each baseline is a configuration flag over one codebase; the attribution-relevant credential/proof path is the only thing that differs. Measured values reproduce the pre-registered prediction exactly.

Baseline	Defense in place	Signal read	Tracks executor?	ais
B1	Shared service account	shared credential	undefined	**0.0**
B2	Per-agent identity	execution-time authenticator	yes	**1.0**
B3	+ RFC 8693 delegation	delegation current actor	no	**0.0**
B4	+ tamper-evident log	delegation current actor	no	**0.0**
B5	+ DPoP sender-constraint	re-exchanged current actor	yes	**1.0**

Contributions.

1. **A measured fix.** Baseline 5: DPOP sender-constrained tokens (RFC 9449) implemented by hand, recovering AIS to 1.0 on both topologies (§4, §10).
2. **A real measurement instrument.** A process-boundary recorder using OS process identity, removing v1's thread-name proxy and proven unspoofable by an in-process rename (§3).
3. **A second metric.** The Log Integrity Score (LIS), backed by a real hash-chained signed log, demonstrating that B4 is tamper-evident (LIS = 1.0) yet still mis-attributing (AIS = 0.0) (§5, §10).
4. **A second topology.** A 3-agent re-delegation chain (T2) showing the gap persists at depth 2 (§6).
5. **Real confidence intervals.** A stochastic Bernoulli(p) policy with Wilson 95% intervals and adaptive sample-size escalation, showing the curve shape is invariant to attack frequency (§7, §10).
6. **Stronger pre-registration.** A SHA-256-locked threat model with a CI gate that fails the build on any edit, mechanizing v1's documentary pre-registration (§9).

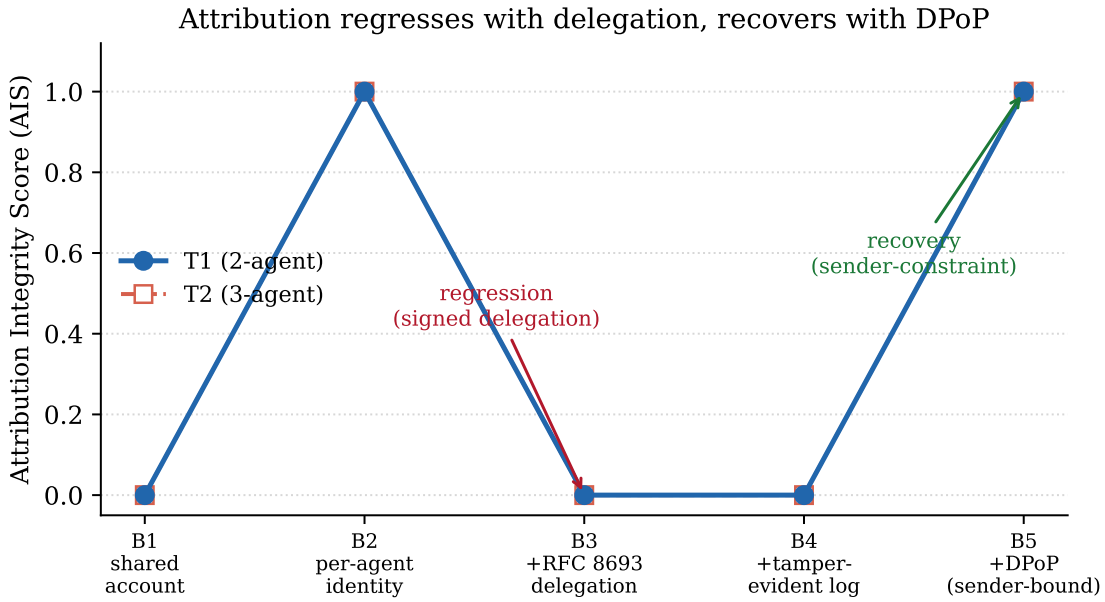

Figure 1: The non-monotonic AIS curve, measured on both topologies. T1 (2-agent) and T2 (3-agent) land on identical points — the gap does not heal with chain depth (§6). Attribution regresses when signed delegation is added (B3) and recovers when sender-constraint is added (B5). Generated from the live harness.

Scope and honesty up front. v2 still concedes: only DPOP (not mutual-TLS, RFC 8705) is measured; two topologies (linear only, no fan-in or cross-organization); scripted deterministic agents (no LLM in the loop); and a synthetic Bernoulli policy rather than real-world attack-frequency telemetry. These are deliberate boundaries defended in §12.

2 Background and Relationship to v1

We assume the v1 background [1]: RFC 8693's `act` claim and its §4.1 current-actor semantics; the confused-deputy lineage from Hardy [13] through Clinejection [11] and the CSA confused-deputy note [10]; the standards call from NIST NCCoE [8] and the OpenID Foundation [9]; and the positioning against *The Misattribution Gap* [12] and SentinelAgent [14]. This section adds only what v2 needs.

2.1 Sender-constrained tokens: DPoP

RFC 8693 inherits OAuth 2.0's default *bearer* holder model: possession of a token is sufficient to use it. DPOP (RFC 9449) [3] binds a token to a key pair the legitimate holder controls. The token carries a confirmation claim $\texttt{cnf} = \{\texttt{jkt} : t\}$ (RFC 7800 [6]), where t is the SHA-256 thumbprint (RFC 7638 [7]) of the holder's public key. On every request the holder presents a short-lived *proof* JWT signed by the private key and bound to the HTTP method and endpoint; the resource verifies the proof and checks that the proof key's thumbprint equals the token's `jkt`. A party that does not hold the private key cannot produce a valid proof, so it cannot wield the token. This is exactly the binding v1 named as the hypothesized fix and did not measure; §4 measures it.

2.2 What is inherited, and the one thing that moved

The adversary model (gray-box; alert-content control only; no token forgery; no process compromise), the attack mechanism (re-delegation misattribution, Path B), the delegation logic, the token-verification logic, and the per-baseline credentials of Baselines 1–4 are inherited unchanged from v1 [1]. The four substrate modules (`auth/tokens.py`, `policy/scope_map.py`, `harness/scorer.py` core, `orchestrator/orchestrator.py`) are reused and extended, never rewritten, so any AIS difference remains attributable to the configuration flag rather than to incidental code differences.

One element of the *execution substrate* did move, and we state it plainly rather than let a reader infer it. In v1 each agent called `siem_action` directly, in its own thread and process. In v2 the agents are separate OS processes and the tool call crosses the process boundary: the agent sends `(command, target, credential, proof)` to the parent harness, which invokes the recorder and `siem_action` on its behalf (§3). The tool therefore executes inside the harness process. This is required for PID-based ground truth and is neutral to the comparison — the verification logic and the presented credential are semantically identical to v1 on the B1–B4 path, and the claimed record still derives solely from the presented token — but it is a real change to the substrate and we record it as a validity threat (§12, item 6).

2.3 OAuth 2.1: the gap is version-independent

The attribution gap is not an artifact of legacy OAuth 2.0. OAuth 2.1 (`draft-ietf-oauth-v2-1-15`, 2 March 2026 [5]) consolidates the core authorization framework — it makes PKCE mandatory, removes the implicit and resource-owner-password grants, and requires exact redirect-URI matching — but it changes neither of the two ingredients this paper's gap depends on. *First*, it does not incorporate or modify RFC 8693 token exchange or the `act` claim, so the requester/executor actor semantics are untouched. *Second*, it retains bearer tokens as the default and makes sender-constraint only a `SHOULD`: "Bearer Tokens *may* be enhanced with proof-of-possession specifications such as DPoP. . . and mTLS" (§1.4.2, "Bearer Tokens"), while authorization and resource servers "*SHOULD* use mechanisms for sender-constraining access tokens" (§1.4.3, "Sender-Constrained Access Tokens"). The fix this paper measures (Baseline 5) is therefore, even under the OAuth 2.1 draft current in 2026, *recommended rather than required* — so the gap is live by default in a fully standards-conformant deployment. AEGIS-AT's authorization core is 2.1-conformant, but the changes 2.1 makes are orthogonal to what we measure: the harness exercises token exchange and direct per-agent credentials, not the implicit, authorization-code/PKCE, or password flows 2.1 revised — so 2.1 is context, not a load-bearing dependency, and is deliberately *not* a separate baseline (it would score identically to B3). As an Internet-Draft, 2.1 is still evolving; should a future revision elevate sender-constraint to a `MUST`, that — and only that — would materially change this framing.

2.4 Proposed bindings, not measured baselines

The multi-agent delegation space has produced numerous binding proposals: Agentic JWT (A-JWT) [15] uses per-agent proof-of-possession keys to prevent in-process impersonation; HDP [17] constructs append-only provenance chains; AIP's Invocation-Bound Capability Tokens [16] report a 100% rejection rate across 600 adversarial attempts; PAuth [18] challenges OAuth's operator-scoped authorization model; and SentinelAgent [14] introduces intent-verified delegation chains for federal systems. Each proposes a mechanism and validates its own scheme. None measures the standardized stack — RFC 8693 alone, RFC 8693 + tamper-evident logging, and RFC 8693 + DPoP sender-constraint — under identical conditions with pre-registered predictions. AEGIS-AT v2 fills this gap:

it is the controlled measurement that exhibits the non-monotonic regression-then-recovery curve, isolating the load-bearing layer that closes the attribution gap.[1]

3 Process-Boundary Ground Truth

v1's recorder read `threading.current_thread().name` as `true_actor` — an honest proxy, but one a misbehaving agent could spoof by renaming its thread. v1 excluded this by its adversary model (alert text only, not agent code); a reviewer is nonetheless entitled to press on it.

v2 removes the proxy. Agents run as `multiprocessing.Process` children spawned by a kernel in the parent harness. The kernel registers the OS-assigned PID → agent-name mapping *at spawn, before the agent's user code runs*, and resolves `true_actor` from that registry. The agent's tool-call message also self-reports its PID, but only so the kernel can cross-check it and fail loud on a mismatch — the resolved identity is always the kernel's, never the message's. The recorder thus reads identity from the kernel's spawn-time PID registry (the PID is OS-assigned), not from any agent-supplied field. The three independence axes from v1 are now *real* rather than argued: a true process boundary (a separate OS process), credential isolation (a per-agent pipe whose child end alone crosses the boundary), and causal precedence (register before serve).

Where the tool runs. Because ground truth is the executing agent's PID observed at the tool boundary, the tool call is mediated by the harness: the agent body sends `(command, target, credential, proof)` over its kernel pipe, and the parent harness runs the recorder and `siem_action` for it, attributing the call to the PID it registered for that pipe. The tool thus executes in the harness process. Two consequences matter for validity. *First*, the agent's private DPOP key never crosses the boundary: the call-time proof is signed *inside* the agent process and only the signed proof is sent (§4). *Second*, nothing the tool verifies or records changes — the verification logic is semantically identical to v1 on the B1–B4 path, the claimed record derives solely from the presented token, and the true record solely from the kernel PID — so centralizing tool execution puts the tool on the same side as the recorder without contaminating the claimed-vs-true comparison. We nonetheless record this substrate change as a validity threat (§12, item 6), as a production deployment would instead run `siem_action` as its own service and observe the caller's authenticated identity at that service boundary — the same construction, relocated.

Spoof resistance, measured. A test agent that renames its thread to `agent:enrich` — the exact move that defeats the v1 recorder — and overwrites its process title obtains the *same* attribution as a clean run on every baseline: B2 stays 1.0 and B3 stays 0.0 with `true_actor` still observed as `contain`. An in-process rename cannot move the call into another process, so PID-based attribution is unmoved. The v1 thread-proxy caveat is retired, and v1's curve reproduces exactly under the subprocess substrate (the regression gate).

4 Baseline 5: Sender-Constrained Tokens (DPoP)

4.1 Mechanism

Each agent is provisioned a DPOP key pair whose private half is used only inside the agent process to sign proofs; the harness generates the keypair and hands the agent its private key (as PKCS8

[1] AgentLeak [19] demonstrates that reproducible multi-agent security benchmarks are a recognized methodological genre; its privacy-leakage focus is orthogonal to our attribution-integrity measurement.

PEM) before the agent's code runs, and only signed proofs — never the private key — cross back over the IPC boundary (made precise in threat-model-v2.1 §A5). A token minted for an agent carries `cnf` = {`jkt` : t} where t is the thumbprint of that agent's public key. The tool requires, on every call: a valid proof JWT signed by the caller's key; equality of the proof key's thumbprint and the token's `jkt`; a fresh `jti` not in a replay cache; and an `iat` within a 60-second window. The orchestrator, when re-delegating, requires a proof from the receiving agent and binds the new token's `cnf` to that agent's key — so an agent cannot obtain a token bound to a key it does not hold. Baselines 1–4 pass `cnf` = NONE and are byte-for-byte the v1 path.

4.2 Why it recovers the curve

Under B1–B4, the token minted naming Enrich is an unbound bearer; Contain lifts it and presents it, and nothing detects the substitution — that lift is the entire attack. Under B5 the token's `jkt` is *Enrich's* thumbprint, and Contain cannot sign a proof under Enrich's private key, so the presentation is rejected before identity is resolved. To act legitimately Contain must obtain its *own* token — bound to Contain's key, naming Contain as current actor, via an execution-time re-exchange. The claimed actor then tracks the executor, and attribution recovers. The replay cache and `iat` window additionally reject a captured proof replayed by another party.

4.3 What the binding does and does not guarantee

The orchestrator verifies *possession*: the presenter holds the private key whose thumbprint equals the token's `jkt`. It does not, on its own, verify that the name written into `act.sub` belongs to that key's holder. In this benchmark the name–key correspondence is established two ways: by construction (the harness mints the executor's token naming the executor and bound to the executor's own freshly generated key), and at execution time (the agent that signs the call-time proof the tool checks is the PID-registered executor, §3). The recovery to AIS = 1.0 follows from the executor wielding a token bound to its own key and naming itself. The distinct variant "an agent obtains a token *naming a different actor* but bound to its *own* key" is principal laundering — a separate attack class v1 and v2 backlog; defending it requires an orchestrator-side name → registered-key registry checked at mint time, named here as the production requirement. The replay cache is a per-run, single-use `jti` set held in the parent harness (no eviction is needed because each run is short and deterministic; a production cache would bound retention to the proof window).

5 Hash-Chained Log and the Log Integrity Score

v1's Baseline 4 was attribution-only: it ran B3's code path, so B4 = B3 by construction. v2 builds the real artifact. Each B4 log entry carries $\mathsf{prev\_hash} = \mathsf{SHA256}(\mathsf{canonical}(\mathsf{entry}) \,\|\, \mathsf{prev\_hash})$, and the chain head is signed by an Ed25519 logger key held only by the parent harness — no agent, the orchestrator, or the tool holds it. A post-hoc rewrite is detectable two ways: the broken chain link, and a head signature that no longer verifies.

The Log Integrity Score. LIS is a metric distinct from AIS:

$$\text{LIS}(B) = \frac{\#\,\text{tampered entries detected by chain verification}}{\#\,\text{tampered entries}}.$$

Under tamper injection (the reported experiments), a baseline with no integrity artifact (B1–B3, B5) has no chain to verify, so a rewrite is undetectable: LIS is 0.0 *by definition* rather than undefined

— the question "was the tamper detected?" has a well-defined answer (no), because a log with no integrity mechanism detects nothing. For B4, every injected tamper is caught and LIS = 1.0.

The load-bearing asymmetry. AIS and LIS are orthogonal: a baseline can score LIS = 1.0 (tamper-evident) and AIS = 0.0 (records the wrong actor) simultaneously, which is exactly B4 (Figure 2). The wrong actor is committed upstream at minting, before any logging layer sees the entry, so a perfect tamper-evident log faithfully preserves a wrong record. This is v1's negative result, now backed by a real cryptographic artifact rather than an asserted equality.

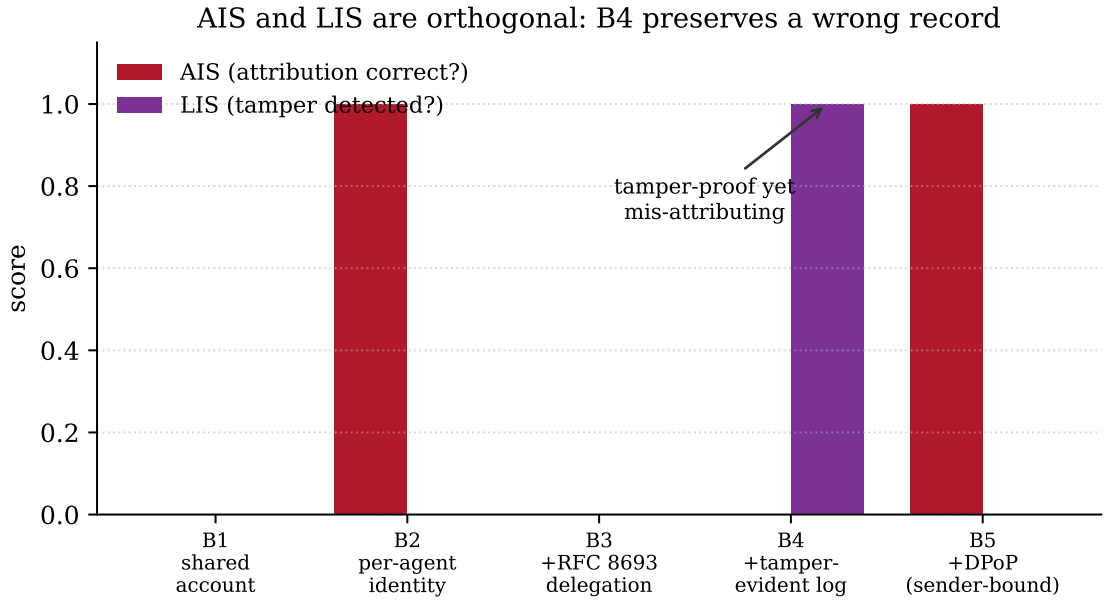

Figure 2: AIS and LIS per baseline. Only B4 carries a hash-chained log, so only B4 reaches LIS = 1.0; yet B4's AIS is 0.0 — the tamper-evident log preserves the wrong record. Generated from the live harness.

6 Topology T2: A Three-Agent Chain

v1's central conceded limitation was its single topology. v2 adds a second, T2: a linear re-delegation chain `Agent-Enrich` → `Agent-Investigator` → `Agent-Contain`, two re-delegation hops. Investigator is a mid-chain agent that receives Enrich's escalation, makes its own correct judgment, and re-delegates containment onward; Contain still executes. We choose a linear chain over a fan-in because linear isolates the chain-*depth* question without the concurrent-write log-ordering confounds a fan-in would add.

The measured T2 curve is identical to T1 (Figure 1; Table 1). At B3/B4 on T2 the claimed actor is the *deepest* requester, Investigator, and the claimed principal chain has grown a hop to [`investigator`, `enrich`, `analyst`] — but the true actor is still the executor, Contain. The gap therefore does not heal with depth; it merely names a different non-executor. To foreclose an unregistered-choice objection we pin the per-hop scope: the root token carries `siem:read siem:write`, and *every* re-delegation hop (Enrich, then Investigator) narrows to `siem:write` — it must, or Contain could not execute `isolate_host`. So at B3/B4 on T2 the claimed and true scope

both equal `siem:write` (scope *matches*, as in T1); the defect is on `actor` and `principal_chain` only. Investigator's standing "read-plus-correlate" role does not change the scope of the containment authority it re-delegates, and giving that token `siem:write` does not prevent the misattribution (scope agreement is orthogonal to actor agreement). B5 is topology-independent by construction: Contain's own bound token is rooted directly at the analyst regardless of the attack path's depth, so B5 recovers to 1.0 on T2 as well. This moves the generalization claim from a structural argument to a result shown on two topologies.

7 Stochastic Policy and Confidence Intervals

v1 reported AIS $\in \{0, 1\}$ from a single canonical execution, so its confidence intervals were degenerate. v2 makes the escalation policy stochastic: Enrich escalates with probability $p \in \{0.1, 0.5, 0.9\}$, and we run N trials per (topology, baseline, p) cell, reporting a Wilson 95% interval on the resulting fraction. Cells whose interval half-width exceeds 0.1 at $N = 100$ are automatically re-measured at $N = 500$ (adaptive escalation), which the low-frequency boundary cells (e.g. $p = 0.1$) trigger.

An exact, not sampled, evaluation. Per-action attribution correctness is deterministic given (baseline, topology) — v1's determinism check proves a run is byte-reproducible, and the pre-registered claim is that p scales the *number* of adversarial actions, not the per-action correctness. We therefore evaluate the attack once per cell and draw N seeded Bernoulli(p) escalation events to form the denominator: re-running the identical deterministic subprocess N times would change nothing but wall-clock. The randomness that matters — which trials escalate — is seeded and reproducible, so the entire grid is a deterministic function of one base seed. That seed is fixed and recorded in the artifact (`base_seed = 20260610`); the point predictions are seed-independent (structural), while the exact interval bounds are a deterministic function of the fixed seed, so the bounds reported below are reproducible exactly. We report two denominators per cell (§10): the adversarial-trigger denominator (escalated trials, $\sim \mathrm{Bin}(N, p)$) and the expanded/structural denominator (all N re-delegated containments). The two yield the same point AIS — itself a manifestation of the structural nature of the gap — with the expanded denominator giving tighter intervals (its size is always N).

8 Implementation

v2 reuses v1's four substrate modules unchanged in spirit and adds, over the same package:

`auth/dpop.py`
: The DPOP primitive: Ed25519 key pairs, proof JWT construction (`htm`/`htu`/`iat`/`jti`), RFC 7638 thumbprints, and server-side proof verification (signature, binding, replay, freshness). A leaf module (Ed25519 only) so an agent subprocess can import it without the RSA-key-generating token module.

`harness/agent_proc.py`, `agent_bodies.py`
: The process-boundary kernel and the bodies that run inside agent subprocesses; identity is the kernel's PID registry, cross-checked against the message's self-reported PID (fail loud on mismatch).

`harness/tamper_log.py`
: The hash-chained, logger-signed tamper-evident log; `verify` returns the indices of broken entries.

`harness/stochastic.py`
: The Bernoulli(p) sweep with Wilson intervals and adaptive sample-size escalation.

`topologies/`
: T1 and T2 as data (a re-delegation hop list); the sweep builds tokens from the declaration, so adding a topology is adding data, not forking code.

`harness/scorer.py`
: Extended with `score_lis` alongside `score_ais`; the AIS path is unchanged.

The token, scope-map, tool, and orchestrator modules gain only an optional, defaulted `cnf`/proof path that is inert on Baselines 1–4.

9 Experimental Methodology

Pre-registration, now mechanized. Every predicted value — the B1–B5 curve on both topologies, the LIS curve, and the stochastic point predictions — is committed in `threat-model-v2.md` before the measuring code. v2 hardens v1's documentary pre-registration into a build invariant: the threat model is hashed (SHA-256 over LF-normalized bytes) into `threat-model-v2.sha256`, and a CI test recomputes the hash and fails the build on any drift. The original file is never edited; clarifications and amendments go in versioned files (`threat-model-v2.1.md`), each with its own lock that the same CI test guards, so the amendment record is as tamper-evident as the original. The v2.1 amendment records the disclosures surfaced in external review (the tool-execution-context change, the precise DPOP binding guarantee, the pinned per-hop scope, the fixed stochastic seed) *without altering any pre-registered prediction.* A contradicted prediction is reported as a finding, not reconciled.

Determinism and reproducibility. Determinism is a checked property on both topologies: each baseline yields byte-identical records across repeated runs under a fixed clock, and a fault-injection test confirms the check catches a drifting clock. The stochastic grid is reproducible from a single base seed. Figures are generated from the live harness, so a plotted number cannot diverge from the code. The suite is **74 v2 tests** plus the **59 frozen v1 tests**; both gates exit 0.

10 Results

10.1 The two curves

The measured AIS curve reproduces the pre-registered prediction exactly on both topologies (Table 1, Figure 1): $B1$=0.0, $B2$=1.0, $B3$=0.0, $B4$=0.0, $B5$=1.0, identical for T1 and T2. `is_non_monotonic` returns `True` on both. The B3 defect breakdown confirms the mechanism rather than an accidental zero: on T1 the claimed actor is `agent:enrich`, on T2 it is `agent:investigator` (the deepest requester), and on both the true actor is `agent:contain`, with the `actor` and `principal_chain` fields mismatched together and `scope` matching. B5's recovery is likewise mechanistic: the lifted token is rejected and the executor re-exchanges for its own bound token.

10.2 Log integrity

The LIS curve is 0.0 for B1, B2, B3, B5 and **1.0** for B4 (Figure 2): only the hash-chained log detects a post-hoc rewrite, and it detects every one. B4's AIS remains 0.0 in the same runs — the tamper-evident log preserves the wrong record. This is the §6.3 asymmetry realized empirically.

10.3 Stochastic confidence intervals

The point AIS equals the pre-registered bit in every cell regardless of p and topology (`curve_shape_invariant` returns `True`); p scales the denominator, not the correctness. Table 2 shows representative T1 cells under *both* denominators of §7: the adversarial-trigger denominator (escalated trials only) and the expanded denominator (all N re-delegated containments, threat-model-v2.md §4.3). The two give identical point estimates — confirming the threat-model §4.3 prediction that the misattribution is latent in the re-delegation pattern, not created by the attacker — while the expanded denominator's intervals are tighter because its size is always N. The low-frequency cells auto-escalate to $N = 500$. Figure 3 plots the full grid.

Table 2: Representative stochastic cells (topology T1), Wilson 95% intervals under both denominators. "esc./N" is the realized escalation count over the trial count; † marks cells that auto-escalated from $N = 100$ to $N = 500$ because the $N = 100$ adversarial half-width exceeded 0.1. The point AIS is identical under both denominators (threat-model §4.3); the expanded interval is tighter (denominator = N). All values are reproducible from the fixed base seed `20260610`.

B	p	**ais**	**adversarial 95% CI**	**expanded 95% CI**	**esc./N**
B2	0.1	1.0	[0.935, 1.000]	[0.992, 1.000]	55/500†
B2	0.5	1.0	[0.930, 1.000]	[0.963, 1.000]	51/100
B2	0.9	1.0	[0.961, 1.000]	[0.963, 1.000]	95/100
B3	0.5	0.0	[0.000, 0.062]	[0.000, 0.037]	58/100
B5	0.1	1.0	[0.839, 1.000]	[0.963, 1.000]	20/100
B5	0.9	1.0	[0.960, 1.000]	[0.963, 1.000]	92/100

11 Discussion

Sender-constraint is the layer that closes the gap. v1 named DPOP as the hypothesized fix and declined to claim it worked without measuring it. v2 measures it: B5 recovers AIS to 1.0 on both topologies. The mechanism is precise — the lift that constitutes the attack is rejected because Contain cannot prove possession of Enrich's key, forcing an execution-time re-exchange that names the executor. This converts v1's "named fix" into a measured one and localizes the missing layer exactly: not "log a different field" (there is no such field) but "bind the token to the executor's key and require an execution-time re-exchange that names the executor."

The robustness changes strengthen, not weaken, the v1 finding. A skeptic could have attributed v1's result to its thread-name proxy, its stub Baseline 4, its single topology, or its degenerate intervals. v2 removes all four: a real process boundary (spoof-resistant), a real hash-chained log (B4 LIS= 1.0 yet AIS= 0.0), a second topology (gap persists at depth 2), and real Wilson intervals (shape invariant across p). Each could have contradicted v1; none did. The gap survives every hardening.

Tamper-evidence and chain depth are orthogonal to attribution. The LIS result makes concrete what v1 argued: a cryptographically perfect audit log can certify the integrity of a wrong record. The T2 result makes concrete that adding hops does not help: each additional requester is one more non-executor the log can name. Both confirm that the only axis that moves AIS is whether

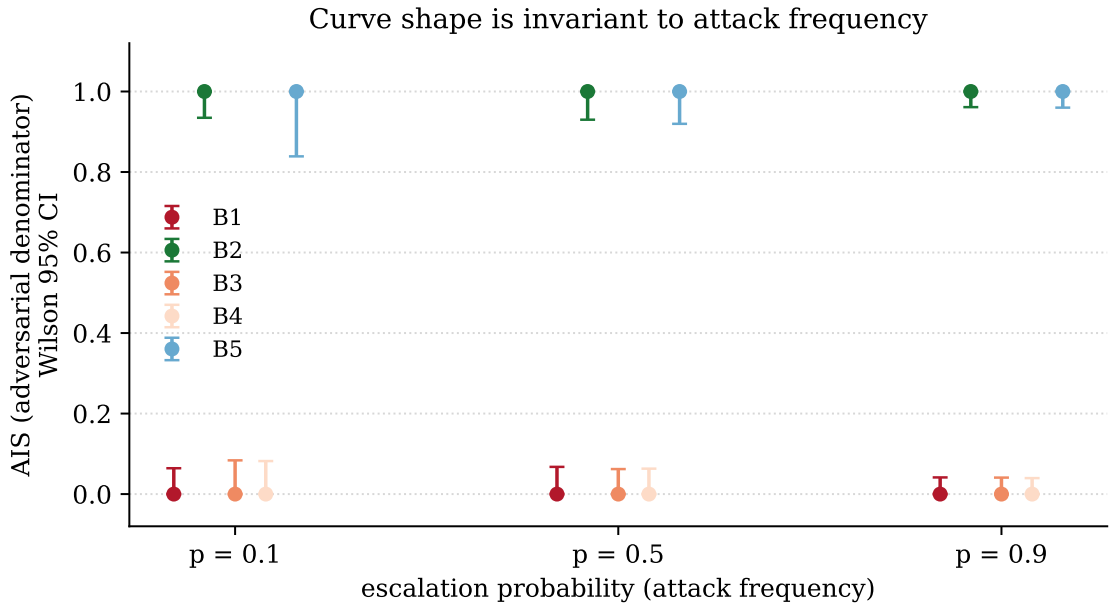

Figure 3: Stochastic AIS with Wilson 95% intervals across escalation probability p (topology T1, adversarial denominator). The point estimates are flat across p — the curve shape is invariant to attack frequency — while the intervals tighten as p (and the escalation count) grows. Generated from the live harness.

the recorded identity is bound to the executor — which is what B2 (authenticator = executor) and B5 (sender-constraint) achieve and B1/B3/B4 do not.

12 Limitations and Validity Threats

1. **DPoP only, not mutual-TLS.** v2 measures sender-constraint via DPOP (RFC 9449). Mutual-TLS-bound tokens (RFC 8705) are a second standardized binding; they require TLS infrastructure and are predicted to behave identically (both close the unbound-bearer gap). Measuring them is a v3 "Baseline 5b."

2. **Two topologies, linear only.** T1 and T2 are linear chains. Fan-in (two agents re-delegating into one executor) and cross-organization delegation each add confounds — concurrent-write log ordering, multiple trust domains — and are deferred to v3.

3. **Scripted agents.** As in v1, the agents are deterministic scripts, which isolates the delegation layer from model behavior but does not measure interaction with imperfect LLM decisions. An optional LLM adapter is deferred to avoid an API-key dependency in the reproducible core.

4. **Synthetic policy.** The Bernoulli(p) escalation is a controlled synthetic policy, not real-world attack-frequency telemetry. Real distributions await an industry partner.

5. **Independence by construction.** The recorder's independence is now a real process boundary rather than a thread proxy, but it is still established by construction and property-based testing, not formal verification.

6. **Tool execution is centralized in the harness.** To observe the executing agent's PID at the tool boundary (§3), the agent sends its tool call over IPC and the parent harness runs `siem_action` for it, so the tool executes in the harness process rather than the agent's — a change from v1, where the agent's own thread called the tool. We argue this does not affect the result: the attack's injection, minting, and hand-off all precede the tool call; the tool's verification logic and the presented credential are semantically identical to v1 on the B1–B4 path; the agent's DPOP private key never crosses the boundary (the proof is signed in the agent process); and the claimed record still derives solely from the presented token while the true record derives solely from the kernel PID. What moved is the host process of an unchanged computation. A production deployment would run `siem_action` as its own service and authenticate the caller at that service boundary — the same construction, relocated. This substrate change is also recorded in the locked threat-model amendment (`threat-model-v2.1.md` §A1).

13 Future Work

- **Baseline 5b — mutual-TLS.** Measure RFC 8705 certificate-bound tokens alongside DPOP; confirm the predicted equivalence.
- **Fan-in and cross-organization topologies.** Move from two linear topologies to aggregation and multi-trust-domain delegation.
- **LLM-in-the-loop.** An optional adapter that plugs a frontier model in as Enrich or Contain, demonstrating the gap holds when the escalating agent is a real LLM, as an appendix demonstration outside the reproducible core.
- **Real telemetry.** Replace the synthetic Bernoulli policy with observed escalation frequencies from a partner SOC deployment.
- **Other sibling-impersonation variants.** Delegation forgery, scope-attenuation bypass, principal laundering — each a distinct mode.

14 Conclusion

AEGIS-AT v1 showed that RFC 8693 delegation can make multi-agent audit attribution worse, not better. v2 asked whether the standardized fix recovers it and whether the finding survives hardening. It does and it does: sender-constrained tokens (DPOP) recover the Attribution Integrity Score to 1.0 at Baseline 5, while a real process boundary, a real hash-chained log, a second topology, and real confidence intervals each leave the non-monotonic curve standing. The Log Integrity Score makes the negative result concrete — a tamper-evident log of the wrong actor — and the three-agent chain shows the gap does not heal with depth. The attribution gap is structural; tamper-evidence and chain depth are orthogonal to it; and binding the token to its holder is the layer that closes it. As standards bodies make RFC 8693 the backbone of AI agent non-repudiation, v2 is a concrete demonstration of both the gap and the specific standardized layer that closes it.

Reproducibility and Artifact Availability

The complete v2 benchmark — the SHA-256-locked threat model, the new and extended modules, the harness, and 74 tests — is released alongside the frozen v1 artifact (tag `v1.0.0`). Code is licensed Apache-2.0; documentation CC BY 4.0. Every AIS, LIS, and stochastic value is asserted in

the test suite against a prediction locked before the measuring code was written, and the figures regenerate from the live harness via `python Documents/Paper/figures/make_figures.py`. The pipeline reproduces in seconds.

References

[1] R. Singh, *AEGIS-AT: Measuring Attribution Integrity Under Sibling Impersonation in Multi-Agent Delegation* (v1), 2026. Frozen artifact, tag `v1.0.0`. https://github.com/rahulsingh1397/AEGIS-AT.

[2] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, *OAuth 2.0 Token Exchange*, RFC 8693, IETF, January 2020. https://www.rfc-editor.org/rfc/rfc8693.

[3] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, *OAuth 2.0 Demonstrating Proof of Possession (DPoP)*, RFC 9449, IETF, September 2023. https://www.rfc-editor.org/rfc/rfc9449.

[4] B. Campbell, J. Bradley, N. Sakimura, and T. Lodderstedt, *OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens*, RFC 8705, IETF, February 2020. https://www.rfc-editor.org/rfc/rfc8705.

[5] D. Hardt, A. Parecki, and T. Lodderstedt, *The OAuth 2.1 Authorization Framework*, Internet-Draft draft-ietf-oauth-v2-1-15, IETF, 2 March 2026 (work in progress; expires 3 September 2026). https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1.

[6] M. Jones, J. Bradley, and H. Tschofenig, *Proof-of-Possession Key Semantics for JSON Web Tokens (JWTs)*, RFC 7800, IETF, April 2016. https://www.rfc-editor.org/rfc/rfc7800.

[7] M. Jones and N. Sakimura, *JSON Web Key (JWK) Thumbprint*, RFC 7638, IETF, September 2015. https://www.rfc-editor.org/rfc/rfc7638.

[8] H. Booth, W. Fisher, R. Galluzzo, and J. Roberts, *Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization*, NIST NCCoE Concept Paper (Initial Public Draft), February 5, 2026. https://www.nccoe.nist.gov/publications/other/accelerating-adoption-software-and-ai-agent-identity-and-authorization-concept.

[9] OpenID Foundation, *OIDF Responds to NIST on AI Agent Security*, March 11, 2026. https://openid.net/oidf-responds-to-nist-on-ai-agent-security/.

[10] Cloud Security Alliance AI Safety Initiative, *Confused Deputy Attacks on Autonomous AI Agents* (Research Note), March 23, 2026. https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-agent-confused-deputy-prompt-injection/.

[11] Snyk, *How "Clinejection" Turned an AI Bot into a Supply Chain Attack*, February 2026. https://snyk.io/blog/cline-supply-chain-attack-prompt-injection-github-actions/.

[12] T. Ahad, I. Hossain, M. J. Alam, S. Puppala, S. B. Alam, and S. Talukder, *The Misattribution Gap: When Memory Poisoning Looks Like Model Failure in Agentic AI Systems*, arXiv:2605.22842, May 2026. https://arxiv.org/abs/2605.22842.

[13] N. Hardy, *The Confused Deputy (or why capabilities might have been invented)*, ACM SIGOPS Operating Systems Review, 22(4), 1988.

[14] K. S. R. Patil, *SentinelAgent: Intent-Verified Delegation Chains for Securing Federal Multi-Agent AI Systems*, arXiv:2604.02767, April 2026. https://arxiv.org/abs/2604.02767.

[15] A. Goswami, *Agentic JWT: Secure Delegation for Agentic AI*, arXiv:2509.13597, September 2025. https://arxiv.org/abs/2509.13597.

[16] S. Prakash, *AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A*, arXiv:2603.24775, March 2026. https://arxiv.org/abs/2603.24775.

[17] A. Dalugoda, *HDP: A Lightweight Cryptographic Protocol for Human Delegation Provenance in Agentic AI Systems*, arXiv:2604.04522, April 2026. https://arxiv.org/abs/2604.04522.

[18] R. K. Sharma, L. Jiang, Z. Lin, and S. Chen, *PAuth: Precise Task-Scoped Authorization for Agents*, arXiv:2603.17170, March 2026. https://arxiv.org/abs/2603.17170.

[19] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, *AgentLeak: A Full-Stack Benchmark for Privacy Leakage in Multi-Agent LLM Systems*, arXiv:2602.11510, February 2026. https://arxiv.org/abs/2602.11510.